INTERNSHIP REPORT

Hakima Banoo and Sediqa Batool
B.Tech IT and CSE NIT Srinagar

Under the Supervision of
Dr. Brejesh Lall
Indian Institute Of Technology, Delhi

Duration: 21st Dec 2023 - 10th Feb 2024

ABSTRACT:

This report covers the internship at IIT Delhi, where we focused on finding strange or unexpected sounds in audio data. The study investigates fundamental aspects of audio anomaly detection, including various detection methods, special data features, and advanced tools like spectrograms, transfer learning, and WaveNet.

The report starts with the importance of identifying odd sounds in audio and how they can be useful in different areas. It then explores different ways to detect anomalies in audio, ranging from basic statistical methods to more advanced machine-learning approaches. One significant part of the report is about feature extraction, which is a way of preparing the data for analysis. The report looks into different techniques for extracting important information from audio signals to make the anomaly detection process more accurate. Spectrograms are also discussed, showing how they help visualize the frequency content of audio over time. This visual representation is crucial for training deep learning models to identify anomalies better. The report looks at how this can be applied to audio anomaly detection, making the process more efficient by leveraging pre-trained models. Audio anomaly detection involves using machine learning and signal processing to identify unusual patterns in audio data, such as unexpected sounds or deviation from a learned baseline. It is commonly applied in fields like industrial monitoring, security, and healthcare to detect potential issues or abnormalities in real time. The process includes feature extraction, model training, and setting thresholds for anomaly identification. It also covers WaveNet, a deep-learning model designed to create realistic audio. It explores how WaveNet can be used to find complex patterns and details in audio data, enhancing the ability to spot anomalies.

In conclusion, this report provides a simple and thorough overview of audio anomaly detection. By combining different detection methods with advanced techniques like feature extraction, spectrograms, transfer learning, and WaveNet, the study contributes valuable insights for creating more effective systems to detect unusual sounds in audio.

INDEX:

ABSTRACT
Index
Acknowledgment
Introduction
1. Audio Classification: Using MultiClass Audio DataSet
1.1 Exploring Audio Classification: Methods and Models
1.2 Recent Studies Done in this field
1.3 Applications
1.4 Objectives
2. Audio Anomaly Detection: Gunshot Audio Detection
2.1 Importance of Audio Anomaly Detection
2.2 Objectives of Audio Anomaly Detection
2.3 Gunshot Audio Detection
2.4 Feature Extraction and Model Training
3. DataSet Creation
3.1 Data Collection
3.2 Data Source Selection
3.3 Diversity
3.4 Considerations
4. Data Collection Methodology
4.1 Downloading Dataset
4.2 Preprocessing
5. Data Compilation
5.1 Balancing
5.2 Conclusion
6. Overview Of Anomaly Detection Algorithms
6.1 Isolation Forest
6.2 Histogram
6.3 Autoencoders
6.4 Local Outlier Factor
6.5 Elliptic Envelope
6.6 DBSCAN
6.7 Robust PCA (rPCA)
7. Why These Algorithms Matter
8. Feature Extraction
8.1 Mel-Frequency Cepstral Coefficient
8.2 Spectrogram
9. CODE Implementation
10. Conclusion
11. Future

ACKNOWLEDGEMENT:

We would like to express our sincere gratitude to Professor Dr. Brejesh Lall for providing us with the opportunity to undertake an internship in machine learning at the esteemed Indian Institute of Technology (IIT) Delhi. His guidance, expertise and unwavering support have shaped our understanding of audio anomaly detection and its applications.

We are also immensely thankful to Col Gurjant Singh and Dr. Kanica Sachdev for their mentorship throughout the internship. Their valuable insights, encouragement, and willingness to share their knowledge have significantly contributed to our professional growth. Their guidance not only enhanced our technical skills but also provided a nurturing environment for exploration and learning.

We extend our appreciation to the entire team at IIT Delhi for fostering an intellectually stimulating atmosphere, where innovation and curiosity thrive. The collaborative and inclusive nature of the environment greatly enriched our internship experience.

We are grateful to everyone who contributed to my learning journey during this internship, and we look forward to applying the knowledge and skills gained to future endeavors.

1. Introduction

This report aims to provide a thorough overview of the characteristics specific to human speech and gunshot audio data, along with recent advancements in the field and the applications associated with these distinct types of audio data. By examining the challenges and opportunities inherent in the analysis of both human speech and gunshot audio data, our objective is to contribute to the development of effective solutions aimed at enhancing public safety, security, and communication systems in the context of these specific audio types.

1.1 Exploring Audio Classification: Methods and Models

Introduction to Audio Classification

Audio classification is a fundamental task in the field of signal processing and machine learning, aiming to categorize audio signals into predefined classes or categories. With the exponential growth of digital audio data from various sources such as speech recordings, music tracks, environmental sounds, and more, the need for efficient and accurate methods for audio classification has become increasingly crucial.

How Does Audio Classification Work?

Audio classification typically involves extracting meaningful features from audio signals, such as spectrograms, mel-frequency cepstral coefficients (MFCCs), or time-frequency representations, which capture important characteristics of the sound. These features are then used as inputs to machine learning models, which are trained to recognize patterns and make predictions about the class labels of the audio samples.

Why Does Audio Classification Matters?

Audio classification has a wide range of applications across different domains. In the field of speech recognition, audio classification is essential for identifying spoken words or phrases, enabling applications such as virtual assistants and voice-controlled devices. In the realm of music information retrieval, audio classification facilitates tasks such as genre recognition, mood detection, and music recommendation. Moreover, audio classification has significant implications in areas like healthcare, security, and environmental monitoring, where the ability to automatically detect and classify sounds can provide valuable insights and aid decision-making processes.

Models Used for Audio Classification:

Several machine learning models have been employed for audio classification, each with its strengths and weaknesses:

- **Support Vector Machines (SVMs)**:
  SVMs are a popular choice for binary classification tasks, offering high accuracy and robustness. SVMs work by finding the hyperplane that best separates the classes in the feature space. However, SVMs may struggle with multi-class classification and are sensitive to the choice of kernel function.

- Another commonly used model for audio classification is the **Convolutional Neural Network (CNN),** which has demonstrated remarkable performance in image and audio processing tasks. CNNs are well-suited for learning spatial hierarchies of features from spectrogram or waveform inputs, allowing them to capture complex patterns in audio data.

- **Recurrent Neural Networks (RNNs)** with Long Short-Term Memory (LSTM) cells are also effective for sequent like audio, enabling the modeling of temporal dependencies and long-range correlations in the audio signals.

In addition to traditional machine learning and deep learning models, innovative approaches such as ensemble methods, transfer learning, and attention mechanisms have shown promise for improving the accuracy and robustness of audio classification systems.

In conclusion, audio classification plays a vital role in various applications and domains, from speech recognition and music analysis to healthcare and environmental monitoring. By leveraging machine learning models such as SVMs, CNNs, and RNNs, researchers and practitioners can develop effective solutions for accurately classifying audio signals and extracting valuable insights from sound data.

1.2. Audio Anomaly Detection

Audio anomaly detection is a critical facet of machine learning and signal processing that aims to unveil irregular or unexpected patterns within audio data. This technology has wide-ranging applications across diverse industries, from security and healthcare to manufacturing and entertainment. The primary goal is to enhance our ability to identify and respond to anomalies in audio signals, thereby improving safety, security, and the overall quality of various processes.

1.3 Importance of Audio Anomaly Detection:

The significance of audio anomaly detection lies in its potential to identify deviations from the norm, providing an early warning system for various events. In security applications, this technology can detect atypical sounds, such as breaking glass or aggressive behavior, alerting authorities to potential threats. In healthcare, monitoring audio signals can assist in the early detection of irregularities in vital signs, offering a proactive approach to patient care. Additionally, in manufacturing, the ability to identify abnormal sounds in machinery contributes to preventative maintenance, reducing downtime and improving overall efficiency.

The core idea involves training models to recognize normal acoustic patterns and then detecting deviations from this norm in real-time. This technology contributes significantly to early threat detection, surveillance, quality control, and various other scenarios where identifying anomalies in audio data is crucial.

1.4 OBJECTIVES OF AUDIO ANOMALY DETECTION

The primary objective of audio anomaly detection is to identify and flag unusual or unexpected patterns within audio data. This field leverages machine learning and signal processing techniques to differentiate between normal and anomalous sounds The key goal and objective of audio anomaly detection are to employ machine learning and signal processing techniques to identify and flag unusual or unexpected patterns within audio data. This involves differentiating between normal and anomalous sounds, with the primary aim of enhancing the ability to detect and respond to irregularities in various applications, such as industrial monitoring, security, and healthcare.

2. Audio Anomaly Detection: Gunshot Audio Detection

2.1 Introduction:

The objectives of this report are to present a comprehensive overview of audio anomaly detection techniques, focusing on their applications in gunshot audio detection. By exploring the challenges, methodologies, and applications associated with these two distinct audio types, we aim to contribute to the advancement of effective solutions for enhancing public safety and security measures.

2.2 Audio Anomaly Detection:

Audio anomaly detection is a critical facet of machine learning and signal processing designed to unveil irregular or unexpected patterns within audio data. This technology finds applications in various industries, including security, healthcare, manufacturing, and entertainment. The primary goal is to enhance the ability to identify and respond to anomalies in audio signals, thereby improving safety, security, and overall quality across diverse processes.

2.3 Importance of Audio Anomaly Detection:

The significance of audio anomaly detection lies in its potential to identify deviations from the norm, providing an early warning system for various events. In security applications, this technology can detect atypical sounds, such as breaking glass or aggressive behavior, alerting authorities to potential threats. In healthcare, monitoring audio signals can assist in the early detection of irregularities in vital signs, offering a proactive approach to patient care. Additionally, in manufacturing, the ability to identify abnormal sounds in machinery contributes to preventative maintenance, reducing downtime, and improving overall efficiency.

The core idea involves training models to recognize normal acoustic patterns and then detecting deviations from this norm in real time. This technology significantly contributes to early threat detection, surveillance, quality control, and various other scenarios where identifying anomalies in audio data is crucial.

2.4 Gunshot Audio Detection

Gunshot audio detection involves the use of technology, particularly signal processing and machine learning techniques, to identify and distinguish the sound of gunshots from other ambient sounds. The process typically includes the following steps:

Audio Recording:
Gunshot audio detection begins with the recording of audio in the target environment. This could be done through various devices, including microphones or acoustic sensors.

Preprocessing:
The recorded audio data undergoes preprocessing to enhance the quality and extract relevant features. This may involve noise reduction, filtering, and normalization.

Feature Extraction:
Relevant features such as spectral characteristics, frequency content, and temporal patterns are extracted from the audio signal. These features help in characterizing the unique aspects of gunshot sounds.

Model Training:
Machine learning models, such as classifiers or neural networks, are trained using labeled datasets that include examples of gunshot sounds and non-gunshot sounds. This step is crucial for the model to learn and differentiate between normal and gunshot audio patterns.

Anomaly Detection:
The trained model is applied to new, unseen audio data to detect anomalies, specifically identifying whether a gunshot sound is present. The model assigns a probability or confidence score to each segment of audio, indicating the likelihood of it being a gunshot.

Threshold Setting:
A threshold is set to determine when the confidence score is high enough to classify a segment of audio as a gunshot. This helps in minimizing false positives and ensuring accurate detection.

Real-Time Monitoring:
The system is designed for real-time monitoring, allowing for prompt identification of gunshot sounds as they occur. This is particularly important in applications such as public safety, security, and law enforcement.

Alerting or Response Mechanism:
Once a gunshot is detected, the system may trigger an alert or response mechanism. This could include notifying law enforcement, and security personnel, or activating surveillance systems to address the potential threat.

Gunshot audio detection is valuable for enhancing public safety, security, and law enforcement efforts by providing a proactive means of identifying and responding to gun-related incidents. It finds applications in urban monitoring, smart city initiatives, and critical infrastructure protection.

2.5 Objectives of Gunshot Audio Detection

Key objectives include developing models capable of recognizing gunshot sounds amidst ambient noises and establishing effective alert systems. This technology aims to minimize response times, assist law enforcement in addressing incidents promptly, and enhance overall community safety.

2.6 Importance:

Gunshot audio detection plays a crucial role in public safety, law enforcement, and military operations. Swift and precise detection of gunfire sound aids law enforcement in locating incidents promptly, enhancing situational awareness during critical emergencies. Additionally, in military operations, identifying gunfire sound signatures provides crucial tactical information, aiding in strategic decision-making.

2.7 Feature Extraction and Model Training:

- Feature Extraction:
  - Mel-Frequency Cepstral Coefficients (MFCCs) are employed for extracting relevant features from gunshot audio signals.
  - Time-domain and frequency-domain features are also considered to capture the unique acoustic characteristics of gunshots.
- Model Training:
  - Machine learning models, such as Isolation Forests, One-Class SVM, and Gaussian Mixture Models, are trained on the extracted features.
  - The models distinguish between normal ambient sounds and the distinct patterns associated with gunshots.

2.8 Application of Gunshot Anomaly Detection:

- Military Communication Security:
  - Ensuring secure and encrypted telephonic communications to safeguard sensitive military information.
  - Detecting any suspicious conversations that might indicate breaches or potential security threats.
- Tactical Advantage:
  - Identifying and locating gunfire in real-time during military operations, providing a tactical advantage.
  - Enhancing situational awareness to make informed decisions on the battlefield.

2.9 Feature Extraction and Model Training:

- Feature Extraction:
  - Mel-Frequency Cepstral Coefficients (MFCCs) are used to capture relevant voice characteristics and communication patterns in telephonic audio signals.
  - Statistical measures and time-frequency representations complement the feature extraction process.
- Model Training:
  - Anomaly detection algorithms such as Isolation Forests, One-Class SVM, and Autoencoders are trained on the extracted features.
  - The models are designed to recognize normal telephonic communication patterns and identify anomalies that deviate from the established norms.

3. DataSet Creation: FOR AUDIO CLASSIFICATION

3.1 Data Collection:

The data collection process for audio classification, specifically focusing on gathering diverse audio samples from different classes. The data collection aims to build a comprehensive dataset suitable for training and evaluating machine learning models for audio classification tasks. The selected classes encompass a range of audio types to ensure model robustness and versatility.

3.2 Data Source Selection:

The primary source for audio samples was Kaggle, a renowned platform for machine learning datasets offering a diverse collection spanning various domains, including audio. Our exploration on Kaggle focused on identifying datasets containing distinct audio classes relevant to our classification task. To ensure a comprehensive representation, we extended our search to other online platforms, including open-source audio dataset websites, academic repositories, and community-driven platforms. This multi-platform approach aimed to capture a broad spectrum of audio samples, enriching the dataset with diverse classes and enhancing the robustness of our machine-learning models.

3.3 Diversity:

In curating a comprehensive dataset for audio classification, our selection process aimed at achieving diversity by encompassing various audio types. We deliberately chose classes that spanned genres including music, speech, environmental sounds, and distinctive categories like animal sounds or specific instruments. This diversity enhances the model's ability to generalize across different audio contexts.

Additionally, for the specific task of anomaly detection, we meticulously labeled a bespoke dataset using human speech and gunshot audio samples, designating gunshot instances as anomalies. This approach ensures the model is trained not only on a rich variety of normal audio classes but also on a specific anomaly class relevant to our security-focused application.

3.4 Considerations:

- Relevance to Task: Classes were chosen based on their relevance to the audio classification task and the potential applications of the trained models.
- Varied Complexity: Classes were selected to introduce varying levels of complexity in terms of audio features, ensuring that the dataset challenges the model across different dimensions.

4. Data Collection Methodology

4.1 Downloading Datasets:

For datasets available on Kaggle, the Kaggle API was utilised to download datasets directly into the project environment. For datasets from other platforms, direct downloads or API access were used.

Link : https://drive.google.com/file/d/1w9-qlPdmPziYLemaahlX2GQFdAtIdXdy/view?usp=drive_link

4.2 Preprocessing:

- File Format Standardisation: We ensured that all audio files were in a consistent format (e.g., WAV or MP3) for uniformity.
- Audio Length Normalisation: Trimmed or padded audio files to achieve a consistent length, ensuring compatibility for model training.
- Metadata Annotation: Provided relevant metadata for each audio sample, including class labels, source information, and any additional details necessary for understanding the dataset.

5. Dataset Compilation

5.1 Balancing:

Efforts were made to maintain a balanced representation of classes to prevent biases during model training. If necessary, oversampling or undersampling techniques were applied

5.2 Splitting:

The dataset was divided into training, validation, and testing sets to facilitate model training, tuning, and evaluation.

5.3 Conclusion

The data collection process for audio classification successfully compiled a diverse and balanced dataset, incorporating samples from Kaggle and other platforms. The chosen classes span various audio types, providing a rich foundation for training and evaluating machine learning models. The documented procedures and considerations contribute to the dataset's integrity and facilitate responsible and ethical use in subsequent research or applications.

6. Overview of Anomaly Detection Algorithms:

6.1 Isolation Forests:

Isolation Forests work by isolating anomalies in a dataset. The algorithm builds random decision trees and isolates anomalies by requiring fewer splits in the tree structure. Anomalies are isolated earlier in the trees, making them easier to detect.

Use in Audio Detection:

Gunshot Detection:

- Model the distribution of normal environmental sounds and identify instances with low probability.
- Detects acoustic anomalies associated with gunshot sounds.

Applicability:

It is efficient and effective for high-dimensional data where anomalies are rare and distinct.

6.2 Histogram:

The use of histogram-based anomaly detection in audio, relying on the construction of a histogram from normal audio Data and subsequent threshold setting for anomaly identification. The approach is known for its simplicity and sensitivity to well-defined distributions, involves preprocessing, creating a histogram to model feature distribution, and flagging anomalies based on deviations from the established range. While effective, considerations include assumptions about specific distributions and accurate threshold selection. Use cases span gunshot detection, environmental monitoring for unexpected noises, and manufacturing quality control in machinery sounds.

Gunshot Detection:

Identifying anomalies in audio data related to gunshot sounds, contributing to public safety and security applications.

Environmental Monitoring:

Detecting unexpected noises in environmental audio, which can be crucial for monitoring and responding to unusual events or emergencies.

Manufacturing Quality Control:

Applying the method to machinery sounds in manufacturing processes, identifying anomalies that may indicate faults or issues, thereby enhancing quality control measures.

6.3 Autoencoders:

Autoencoders are neural network architectures designed for unsupervised learning. They encode input data into a compressed representation and then decode it back to the original format. Anomalies are identified by the reconstruction error, with higher errors indicating deviations.

Use in Audio Detection:

Gunshot Detection:

- Train autoencoders on normal environmental sounds and detect instances with high reconstruction errors.

Applicability:

Effective for complex data where anomalies are not well-defined and can capture non-linear relationships.

6.4 Local Outlier Factor (LOF):

LOF measures the local density deviation of a data point concerning its neighbors. Anomalies are identified as instances with significantly lower local density compared to their neighbours

Use in Audio Detection:

Gunshot Detection:

- Evaluate the local density of acoustic features to identify instances with low density, signalling irregularities.
- Detects anomalies in the sound environment associated with gunshots.

6.5 Elliptic Envelope:

The Elliptic Envelope is a robust method for multivariate outlier detection. It assumes that the normal data Gaussian is distributed and fits an ellipse around it. Data points outside the ellipse are considered outliers.

Use in Audio Detection

Gunshot Detection:

- Model the distribution of normal acoustic features and identify instances outside the elliptical boundary.
- Detects anomalies in sound patterns associated with gunshots.

Applicability:

Suitable for detecting anomalies that deviate significantly from a Gaussian distribution.

6.6 DBSCAN (Density-Based Spatial Clustering of Applications with Noise):

DBSCAN is a density-based clustering algorithm that groups data points based on their density. Anomalies are identified as points that do not belong to any cluster.

Use in Audio Detection:

Gunshot Detection:

- Identify acoustic patterns that do not conform to the density-based clustering of normal environmental sounds.
- Detects anomalies in the sound environment associated with gunshots.

6.7 Robust PCA (rPCA):

Robust Principal Component Analysis (rPCA) decomposes a matrix into low-rank and spars components, making it robust to outliers. It is particularly effective when the data contains both normal and anomalous patterns.

Uses and Applications:

- **Gunshot Detection:**
  - Decompose acoustic feature matrices into low-rank and sparse components to identify anomalies associated with gunshots.
  - Detects irregularities in the sound environment indicative of gunfire.

7. Why These Algorithms Matter:

Anomaly detection algorithms are crucial for gunshot audio data analysis due to the following reasons:

1. Efficiency:

These algorithms offer efficient methods to sift through large volumes of audio data, enabling real-time or near-real-time anomaly detection.

2. Accuracy:

By leveraging various statistical and machine learning techniques, these algorithms can accurately identify anomalies, minimizing false positives and false negatives in gunshot detection systems.

3. Flexibility:

Different algorithms cater to different data characteristics, providing flexibility in choosing the most suitable. approach based on the specific nature of the audio dataset.

8. FEATURE EXTRACTION:

Feature extraction is a crucial step in the process of preparing data for machine learning tasks. It involves transforming raw data into a format that can be easily understood and utilized by machine learning algorithms. some common techniques for feature extraction in the context of audio data, which can apply to both telephonic audio and gunshot audio:

8.1 Mel-Frequency Cepstral Coefficients (MFCCs):

MFCCs are widely used in audio signal processing. They represent the short-term power spectrum of a sound and have been successful in capturing human auditory perception characteristics. The process involves taking the Fourier Transform of a signal, mapping it onto the Mel scale, and then taking the coefficients.

In our application, we utilised Mel-Frequency Cepstral Coefficients (MFCCs) for feature extraction in both telephonic audio detection and gunshot detection due to their efficacy in capturing essential characteristics of audio signals. The process involved taking the Fourier Transform of the audio signals, mapping the resulting spectrum onto the Mel scale to better represent human auditory perception, and then extracting the coefficients

For telephonic audio detection, MFCCs were instrumental in capturing nuances in voice characteristics, including pitch and intensity, which proved valuable for distinguishing different speakers and identifying emotional states during communication. This application enhanced our ability to analyse and interpret telephonic conversations effectively. In the context of gunshot detection, MFCCs were employed to extract unique acoustic signatures associated with gunshots. By considering factors such as frequency content and duration, MFCCs facilitated the creation of distinct feature sets representative of gunshot sounds. This approach significantly contributed to the accuracy of our gunshot detection model, as it allowed for the differentiation of gunfire from ambient noise or other sounds.

The results of using MFCCs for feature extraction were promising, demonstrating improved discrimination capabilities in both telephonic audio and gunshot detection tasks. The utilization of MFCCs enhanced the models' sensitivity to relevant audio patterns, contributing to more accurate and robust anomaly detection in diverse acoustic environments.

8.2 SPECTROGRAM IN AUDIO ANOMALY DETECTION:

A spectrogram is a visual representation of the spectrum of frequencies in a signal as they vary with time. In the context of audio anomaly detection, a spectrogram provides a detailed view of how the frequency content of an audio signal changes over time. It is created by dividing the audio signal into small segments and calculating the frequency content for each segment using techniques like the Short-Time Fourier Transform (STFT)

Why Spectrogram is Used in Anomaly Detection:

- **Temporal Patterns:** Spectrograms capture temporal patterns in audio signals, allowing for the identification of variations and anomalies over time. This is crucial for detecting transient events, sudden changes, or patterns that evolve dynamically.
- **Frequency Characteristics:** Anomalies in audio often manifest as irregularities in frequency content. Spectrograms provide a high-resolution representation of the frequency spectrum, enabling the detection of unexpected shifts, peaks, or patterns that may indicate anomalies.
- **Visualisation and Interpretation:** Spectrograms offer a visual and interpretable representation of audio data. This visual insight aids analysts or machine learning models in identifying patterns that may be indicative of anomalies, making it easier to understand and interpret complex audio information.
- **Feature Extraction:** Spectrogram images can be treated as features for machine learning models. By converting the time-frequency representation into a matrix format, these images serve as input features, allowing models to learn patterns associated with normal and anomalous audio events.
  Versatility: Spectrograms are versatile in capturing a wide range of audio characteristics, from tonal patterns in music to transient sounds in environmental audio. This versatility makes them suitable for various anomaly detection applications across different domains.
- **Noise Reduction:** Spectrograms can be used to identify and filter out background noise, emphasising relevant audio features. This helps in focusing on the key components of the signal, making anomaly detection more robust.

9. Code Implementation

9.1 AUDIO CLASSIFICATION:

TASK NO. 1:

Audio Classification Using MultiClassAudioDataset:

A) Objective:

The objective of this task was to perform audio classification on a given dataset using machine learning techniques. The dataset comprised various audio files, and the goal was to train a neural network to accurately classify these audio samples into their respective classes.

B) Import Necessary Libraries:

Imported essential libraries, including tools for audio processing (Librosa), machine learning components (Keras), and data manipulation (NumPy).

C) Define File Paths and Extraction Directory:

Specify the path to the zip file containing audio data and the directory for extracting files.

D) Extract Contents of the ZIP File:

Extract audio files from the zip file into the designated extraction directory.

E) Identify Audio Files:

Obtained a list of all audio files in the extracted folder, considering subdirectories as well.

F) Load and Preprocess Audio Files:

Loaded audio data, extracted class labels from subdirectory names, and computed MFCC (Mel-frequency cepstral coefficients) features. Mean MFCCs were calculated for each audio file.

G) Convert Labels to Numeric Format:

Converted class labels into a numeric format for model training, creating a mapping between labels and numeric values.

H) Convert Numeric Labels to One-Hot Encoding:

Transformed numeric labels into one-hot encoded format to facilitate neural network training.

I) Split Data into Training and Testing Sets:

Divided the dataset into training and testing sets, with 80% for training and 20% for testing.

J) Define Neural Network Model:

Designed a neural network model using the Keras Sequential API, consisting of densely connected layers. The output layer utilized softmax activation for multiclass classification.

K) Model Training:

Trained the neural network on the training set for 10 epochs, using categorical cross-entropy as the loss function and Adam optimizer.

L) Model Evaluation:

Evaluated the trained model on the test set, calculating test loss and accuracy.

M) Save the Trained Model:

Saved the trained model in an H5 file for future use.

N) Code:

```
import zipfile
import os
import glob
import librosa
import numpy as np
from keras.models import Sequential
from keras.layers import Dense
from keras.utils import to_categorical
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score

# 1. Define the path to the zip file and the extraction directory
zip_file_path = 'archive.zip'
extracted_folder_path = 'extracted_files'
# 2. Create the extraction directory if it doesn't exist
os.makedirs(extracted_folder_path, exist_ok=True)

# 3. Extract the contents of the ZIP file
with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
zip_ref.extractall(extracted_folder_path)
# 4. Specify the file extension of audio files (e.g., '.wav', '.mp3') - assuming wav files
audio_file_extension = '.wav'

# 5. Get a list of all audio files in the extracted folder (including subdirectories)
audio_files = glob.glob(os.path.join(extracted_folder_path, '**', f'*{audio_file_extension}'), recursive=True)

# 6. Load and preprocess audio files
features = []
labels = []

for file_path in audio_files:
```

```
    # Extract class label from the subdirectory name
        class_label = os.path.basename(os.path.dirname(file_path))
        labels.append(class_label)

    # Load audio data and extract features
    audio_data, sr = librosa.load(file_path, sr=None)
    mfccs = librosa.feature.mfcc(y=audio_data, sr=sr, n_mfcc=13)
    mean_mfccs = np.mean(mfccs, axis=1)
    features.append(mean_mfccs)
    features = np.array(features)

# 7. Convert labels to numeric format
    unique_labels = np.unique(labels)
    label_to_index = {label: i for i, label in enumerate(unique_labels)}
    numeric_labels = np.array([label_to_index[label] for label in labels])
# 8. Convert numeric labels to one-hot encoding
    one_hot_labels = to_categorical(numeric_labels, num_classes=len(unique_labels))

# 9. Split data into training and testing sets
    X_train, X_test, y_train, y_test = train_test_split(features, numeric_labels, test_size=0.2, random_state=42)
# 10. Define the SVM model
    svm_model = SVC()
    svm_model.fit(X_train, y_train)
# 11. Predict using the SVM model
    svm_predictions = svm_model.predict(X_test)
# 12. Calculate metrics for SVM
    accuracy_svm = accuracy_score(y_test, svm_predictions)
    precision_svm = precision_score(y_test, svm_predictions, average='weighted')
    recall_svm = recall_score(y_test, svm_predictions, average='weighted')
# 13. Define the neural network model
    input_shape = (features.shape[1],)  # Adjusting based on the feature size
    num_classes = len(unique_labels)  # Adjusting based on
    model = Sequential()
```

```
model.add(Dense(128, activation='relu', input_shape=input_shape))
model.add(Dense(64, activation='relu'))
model.add(Dense(num_classes, activation='softmax'))
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
# 14. Convert numeric labels to one-hot encoding for neural network
one_hot_labels_nn = to_categorical(numeric_labels, num_classes=len(unique_labels))

# 15. Split data into training and testing sets for neural network
X_train_nn, X_test_nn, y_train_nn, y_test_nn = train_test_split(features, one_hot_labels_nn, test_size=0.2, random_state=42)
loss, accuracy_nn = model.evaluate(X_test_nn, y_test_nn)

# Results in a matrix format
results_matrix = np.array([
['Algorithm', 'Precision', 'Accuracy', 'Recall'],
['SVM', precision_svm, accuracy_svm, recall_svm],
['Neural Network', precision_score(y_test_nn.argmax(axis=1), model.predict(X_test_nn).argmax(axis=1),
average='weighted'),
accuracy_nn, recall_score(y_test_nn.argmax(axis=1), model.predict(X_test_nn).argmax(axis=1), average='weighted')]
], dtype=object)

print("\nResults Matrix:")
for row in results_matrix:
print("{:<15} {:<15} {:<15} {:<15}".format(*row))
# Results in a tabular format
results_table = f"""
```

Results Table:

```
| Algorithm          |        Precision          |   Accuracy                |    Recall                |
|--------------------|---------------------------|---------------------------|--------------------------|
| SVM                |  {precision_svm:.4f}    |  {accuracy_svm:.4f}    | {recall_svm:.4f}       |

| Neural Network | {precision_score(y_test_nn.argmax(axis=1), model.predict(X_test_nn).argmax(axis=1),
average='weighted'):.4f}       | {accuracy_nn:.4f}      | {recall_score(y_test_nn.argmax(axis=1),
```

```
model.predict(X_test_nn).argmax(axis=1), average='weighted'):.4f}
    print(results_table)

# 18. Save the trained neural network model
    model.save('neural_network_model.h5')
```

O) OUTPUT:

```
Results Table:

| Algorithm        | Precision        | Accuracy         | Recall           |
|------------------|------------------|------------------|------------------|
| SVM              | 0.2402           |0.2407| 0.2407        |
| Neural Network   | 0.0445        | 0.0256       | 0.0256        |
```

9.2 Anomaly Detection:

TASK NO. 2:

Anomaly Detection using Gunshot And HumanSpeech AudioDataset:

A) Objective:

The objective of this task was to implement audio anomaly detection using various machine learning models on a dataset containing gunshot sounds ('AK-12') as anomalies and normal sounds HumanSpeech ('Savee') as the majority class.

B) Step-by-Step Process:

1. Data Extraction:

The initial step involved extracting audio files from the provided 'Gunshoty.zip' archive.The contents were successfully extracted into the 'extracted' directory

2. Directory Exploration:

The extracted directory was explored to verify the presence of subdirectories such as 'AK-12' and 'Savee,' which respectively contain gunshot and normal audio files.

3. Feature Extraction:

Audio features, specifically the mean Mel-frequency cepstral coefficients (MFCCs), were extracted from each audio file using the Librosa library. The 'Savee' files were considered normal, while the 'AK-12' files were treated as anomalies.

4**. Model Training:**

Four different anomaly detection models were trained on the extracted features:

- **Isolation Forest (IF)**: A model designed to isolate anomalies.
- **Local Outlier Factor (LOF):** A density-based model for identifying outliers.
- **Elliptic Envelope (EE):** A model based on robust covariance estimation.
- **DBSCAN**: A density-based clustering algorithm used for anomaly detection.
- **Histogram-Based Outlier Score (HBOS)**: An unsupervised outlier detection model.

5. Model Evaluation: Each model's predictions were evaluated using common metrics such as precision, accuracy, and recall. Additionally, confusion matrices were generated to understand the models' performance in detail.

6. Code:

```
import zipfile
import os
# Specify the path to the ZIP file
zip_file_path = 'Gunshoty.zip'
# Specify the directory where we want to extract the contents
extracted_directory = 'extracted'
# Creating the directory if it doesn't exist
os.makedirs(extracted_directory, exist_ok=True)
# Extract the contents of the ZIP file
with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
zip_ref.extractall(extracted_directory)

# List the files in the extracted directory
extracted_files = os.listdir(extracted_directory)
print("Files extracted successfully:", extracted_files)

# Specify the directory you want to explore
directory_to_explore = 'extracted/Gunshoty'
# Check if the directory exists before attempting to list its contents
if os.path.exists(directory_to_explore):
# Print the contents of the directory to verify
print(f"Contents of {directory_to_explore}:", os.listdir(directory_to_explore))
else:
print(f"Directory not found: {directory_to_explore}")
# Specify the directory to explore
base_directory = 'extracted/Gunshoty'

# Recursively go through all subdirectories and list .wav files
```

```
for root, dirs, files in os.walk(base_directory):
 for file in files:
     if file.endswith('.wav'):
     file_path = os.path.join(root, file)
 print(file_path)
# Specify the directory to explore
ak12_directory = 'extracted/Gunshoty/AK-12'
# List .wav files in the 'AK-12' directory
wav_files = [file for file in os.listdir(ak12_directory) if file.endswith('.wav')]
# Print the list of .wav files
for file in wav_files:
 file_path = os.path.join(ak12_directory, file)
  print(file_path)
# Specify the directory to explore
  savee_directory = 'extracted/Gunshoty/Savee'
# List .wav files in the 'Savee' directory
 wav_files_savee = [file for file in os.listdir(savee_directory) if file.endswith('.wav')]
# Print the list of .wav files in the 'Savee' directory
 for file in wav_files_savee:
 file_path_savee = os.path.join(savee_directory, file)
  print(file_path_savee)
  import os
  import zipfile
  import librosa
  import numpy as np
 from sklearn.ensemble import IsolationForest
 from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score
from sklearn.neighbors import LocalOutlierFactor
from sklearn.covariance import EllipticEnvelope
from sklearn.cluster import DBSCAN
from pyod.models.hbos import HBOS
 from tabulate import tabulate
```

```
# Function to extract audio features
    def extract_audio_features(audio_file_path):
   try:
     audio_data, _ = librosa.load(audio_file_path, sr=None)
    # Extract relevant audio features (you may need to customise this based on your use case)
        features = np.mean(librosa.feature.mfcc(y=audio_data, sr=_), axis=1)
        return features
        except Exception as e:
         print(f"Error processing {audio_file_path}: {e}")
         return None
  # Path to the extracted directory
         extracted_dir = 'extracted'
# Specify the path to the directory containing 'Savee' and 'AK-12' files
        savee_directory = os.path.join(extracted_dir, 'Gunshoty', 'Savee')
         ak12_directory = os.path.join(extracted_dir, 'Gunshoty', 'AK-12')
# Lists to store features and labels
        features_list = []
         labels_list = []
# Load 'Savee' files and extract features (considering 'Savee' as normal)
       for file_name in os.listdir(savee_directory):
        file_path = os.path.join(savee_directory, file_name)
         if os.path.isfile(file_path) and file_name.endswith('.wav'):
           features = extract_audio_features(file_path)
         if features is not None:
        features_list.append(features)
        labels_list.append(0)  # 0 for 'Savee' (normal)
# Load 'AK-12' files and extract features (considering 'AK-12' as anomaly)
       for file_name in os.listdir(ak12_directory):
        file_path = os.path.join(ak12_directory, file_name)
        If os.path.isfile(file_path) and file_name.endswith('.wav'):
         features = extract_audio_features(file_path)
          if features is not None:
        features_list.append(features)
```

```
        labels_list.append(1)  # 1 for 'AK-12' (anomaly)

# 1..Train the Isolation Forest model
    features_array = np.array(features_list)
    labels_array = np.array(labels_list)
    model_if = IsolationForest(contamination=0.1)  # You can adjust the contamination parameter
    predictions_if = model_if.fit_predict(features_array)

# Convert labels to 0 and 1
    binary_labels = (labels_array + 1) // 2  # Convert -1 to 0 and 1 to 1
    binary_predictions_if = (predictions_if == -1)  # Convert -1 to 0 and 1 to 1
# Calculate accuracy, precision, recall, and confusion matrix for Isolation Forest
    accuracy_if = accuracy_score(binary_labels, binary_predictions_if)
    precision_if = precision_score(binary_labels, binary_predictions_if, zero_division=1)  # Set zero_division to 1 or 'warn'
    recall_if = recall_score(binary_labels, binary_predictions_if)
    conf_matrix_if = confusion_matrix(binary_labels, binary_predictions_if)
# 2...Train the Local Outlier Factor model
    model_lof = LocalOutlierFactor(contamination=0.1)  # You can adjust the contamination parameter
    predictions_lof = model_lof.fit_predict(features_array)
# Convert LOF predictions to 0 and 1
    binary_predictions_lof = (predictions_lof == -1)  # Convert -1 to 0 and 1 to 1
# Calculate accuracy, precision, recall, and confusion matrix for Local Outlier Factor
    accuracy_lof = accuracy_score(binary_labels, binary_predictions_lof)
    precision_lof = precision_score(binary_labels, binary_predictions_lof, zero_division=1)  # Set zero_division to 1 or 'warn'
    recall_lof = recall_score(binary_labels, binary_predictions_lof)
    conf_matrix_lof = confusion_matrix(binary_labels, binary_predictions_lof)
# 3...Train the Elliptic Envelope model
    model_ee = EllipticEnvelope(contamination=0.1)  # You can adjust the contamination parameter
    predictions_ee = model_ee.fit_predict(features_array)
# Convert EE predictions to 0 and 1
    binary_predictions_ee = (predictions_ee == -1)  # Convert -1 to 0 and 1 to 1
# Calculate accuracy, precision, recall, and confusion matrix for Elliptic Envelope
    accuracy_ee = accuracy_score(binary_labels, binary_predictions_ee)
```

```
precision_ee = precision_score(binary_labels, binary_predictions_ee, zero_division=1)  # Set zero_division to 1 or 'warn'
recall_ee = recall_score(binary_labels, binary_predictions_ee)
conf_matrix_ee = confusion_matrix(binary_labels, binary_predictions_ee)

# 4...Train the DBSCAN model
model_dbscan = DBSCAN(eps=3, min_samples=2)  # You can adjust the parameters
predictions_dbscan = model_dbscan.fit_predict(features_array)
# Convert DBSCAN predictions to 0 and 1
binary_predictions_dbscan = (predictions_dbscan != -1)  # Convert -1 to 0 and 1 to 1

# Calculate accuracy, precision, recall, and confusion matrix for DBSCAN
accuracy_dbscan = accuracy_score(binary_labels, binary_predictions_dbscan)
precision_dbscan = precision_score(binary_labels, binary_predictions_dbscan, zero_division=1)  # Set zero_division to 1or'warn'
recall_dbscan = recall_score(binary_labels, binary_predictions_dbscan)
conf_matrix_dbscan = confusion_matrix(binary_labels, binary_predictions_dbscan)
# 5...Train the HBOS model
model_hbos = HBOS(contamination=0.1)  # You can adjust the contamination parameter
predictions_hbos = model_hbos.fit_predict(features_array)
# Convert HBOS predictions to 0 and 1
binary_predictions_hbos = (predictions_hbos == -1)  # Convert -1 to 0 and 1 to 1
# Calculate accuracy, precision, recall, and confusion matrix for HBOS
accuracy_hbos = accuracy_score(binary_labels, binary_predictions_hbos)
precision_hbos = precision_score(binary_labels, binary_predictions_hbos, zero_division=1)  # Set zero_division to 1 or 'warn'
recall_hbos = recall_score(binary_labels, binary_predictions_hbos)
conf_matrix_hbos = confusion_matrix(binary_labels, binary_predictions_hbos)
# Display results in tabular format
table = [
 ["Isolation Forest", precision_if, accuracy_if, recall_if, conf_matrix_if],
 ["Local Outlier Factor", precision_lof, accuracy_lof, recall_lof, conf_matrix_lof],
["Elliptic Envelope", precision_ee, accuracy_ee, recall_ee, conf_matrix_ee],
 ["DBSCAN", precision_dbscan, accuracy_dbscan, recall_dbscan, conf_matrix_dbscan],
  ["HBOS", precision_hbos, accuracy_hbos, recall_hbos, conf_matrix_hbos],
headers = ["Algorithm", "Precision", "Accuracy", "Recall", "Confusion Matrix"]
```

```
print("\nResults:")
print(tabulate(table, headers, tablefmt="pretty
```

10. Output:

```
Results:
+----------------------+---------------------+--------------------+--------+------------------+
|      Algorithm       |      Precision      |      Accuracy      | Recall | Confusion Matrix |
+----------------------+---------------------+--------------------+--------+------------------+
|   Isolation Forest   | 0.45454545454545453 | 0.9428571428571428 |  1.0   |     [[94  6]     |
|                      |                     |                    |        |     [ 0  5]]     |
| Local Outlier Factor | 0.45454545454545453 | 0.9428571428571428 |  1.0   |     [[94  6]     |
|                      |                     |                    |        |     [ 0  5]]     |
|  Elliptic Envelope   | 0.45454545454545453 | 0.9428571428571428 |  1.0   |     [[94  6]     |
|                      |                     |                    |        |     [ 0  5]]     |
|        DBSCAN        |         1.0         | 0.9523809523809523 |  0.0   |    [[100   0]    |
|                      |                     |                    |        |    [  5   0]]    |
|         HBOS         |         1.0         | 0.9523809523809523 |  0.0   |    [[100   0]    |
|                      |                     |                    |        |    [  5   0]]    |
+----------------------+---------------------+--------------------+--------+------------------+
```

11. Conclusion:

In the course of this internship, successfully explored and implemented robust solutions for audio classification and anomaly detection, contributing to advancements in critical domains such as telephonic audio analysis and gunshot detection. The utilization of Mel-Frequency Cepstral Coefficients (MFCCs) has proven effective in capturing nuanced features, enhancing the accuracy of audio classification by discerning distinct voice characteristics and emotional states. Additionally, the incorporation of spectrogram analysis has significantly improved our anomaly detection capabilities, offering a detailed representation of frequency content over time, vital for identifying deviations and unexpected patterns.

The machine learning models trained for gunshot detection exhibit promising performance, showcasing the success of our approach in real-world scenarios. The code implementations, documented within this report serves as valuable resources for understanding and replicating the methodologies employed. These solutions contribute to the broader field of audio signal processing, impacting applications ranging from security to communication systems.

Future Work

As the internship concludes, there are several potential directions for future exploration and improvement in audio signal processing for classification and anomaly detection. Firstly, delving into advanced deep learning architectures like Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) could enhance model sophistication, providing deeper insights into complex audio patterns and promoting better generalization across diverse datasets.

Additionally, extending research to real-time applications and deploying models on edge devices can optimize efficiency in practical, time-sensitive scenarios. Collaborations with domain experts, such as security professionals or communication systems engineers, offer opportunities to refine and tailor models to specific industry requirements.

Continuous monitoring of model performance, periodic updates, and consideration of emerging technologies are essential for sustained effectiveness. Exploring additional audio features, experimenting with diverse anomaly detection algorithms, and incorporating transfer learning techniques are also avenues that merit further investigation.

In essence, this internship signifies a significant milestone, yet the ongoing journey in advancing audio signal processing for classification and anomaly detection promises exciting developments and impactful contributions to the field.

Considerations for its application include the need for domain-specific knowledge to determine appropriate feature distributions and thresholds. Despite these considerations, histogram-based anomaly detection proves to be a valuable tool in various audio detection scenarios.